

The Normalization Paradox: Resolving Data Anomalies and Benchmarking Join Costs in a Legacy Hospital Migration

Zachary Frappier

Department of Computer Science
California State University, Northridge
Email: zachary.frappier.132@my.csun.edu

Abstract—The MedCorp Network Merger produced a heavily denormalized 100,000-row hospital dataset containing repeated patient, doctor, ward, appointment, and treatment information. This study reverse-engineers the legacy relation, derives its functional dependencies, and decomposes it into a BCNF-oriented schema. Because the original data does not contain an appointment identifier, a surrogate `Appointment_ID` is generated during reconstruction. Python benchmarks compare the flat and normalized representations using memory footprint, doctor-phone mutation time, and read latency for 1,000 patients. The normalized representation used 93.088 MB compared with 113.056 MB for the flat structure. Median doctor mutation time decreased from 3.807 ms to 0.0001 ms, while the normalized read path decreased from 4.024 ms to 2.264 ms. These results show substantial integrity and update benefits and demonstrate that read performance depends on implementation and indexing strategy.

I. INTRODUCTION

The MedCorp Network Merger combined two hospital information systems into one flat legacy dataset. Patient, doctor, ward, and treatment information is repeated across many rows, producing update, insertion, and deletion anomalies. For example, changing a doctor’s phone number may require modifying thousands of redundant records.

This paper asks whether decomposing the legacy relation into a BCNF-oriented design reduces redundancy and mutation cost while maintaining acceptable read performance. The study contributes an evidence-based ERD, a functional-dependency and normalization proof, a reproducible Python reconstruction pipeline, and an empirical comparison of flat and normalized in-memory structures.

II. BACKGROUND

A. Relational Modeling and Normalization

A relation is a structured collection of tuples and attributes. Primary keys identify records, while foreign keys represent relationships between relations. A functional dependency $X \rightarrow Y$ states that the attribute set X determines the attribute set Y . Normalization uses these dependencies to reduce redundancy and improve consistency. Codd’s relational model established relations, normal forms, and operations intended to address redundancy and consistency in shared data systems [1].

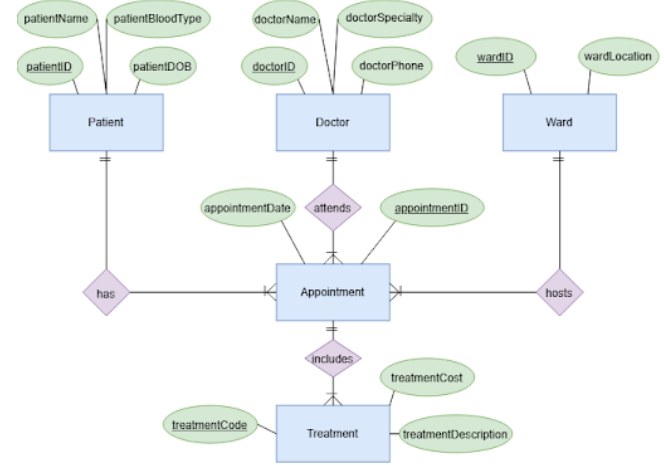


Fig. 1. MedCorp entity-relationship diagram.

B. Modern Normalization and Denormalization

Modern systems may transform normalized relational schemas into denormalized or read-oriented structures when workload requirements justify duplicated data. Denormalization can reduce joins, but it also increases consistency-management responsibilities. Recent research treats denormalization as a workload-sensitive schema-transformation decision. Mali *et al.* present a model-driven approach for generating alternative schemas during migration [2]; Bansal *et al.* use query patterns to guide relational-to-NoSQL denormalization and evaluate query-performance improvements [3].

C. MedCorp Dataset and ERD

The source dataset is one ordered legacy dump. Earlier uploaded “CSV split files” were PDF exports of the original file and were not treated as independent clean relations. The machine-readable dump contains 100,000 rows and 14 columns. The reconstructed entities are Patient, Doctor, Ward, Appointment, Treatment, and the AppointmentTreatment association.

TABLE I
FUNCTIONAL DEPENDENCIES

Determinant	Dependent attributes
Patient_ID	Patient name, DOB, and blood type
Doctor_ID	Doctor name, specialty, and phone
Ward_ID	Ward location
Treatment_Code	Treatment description and cost

TABLE II
RECONSTRUCTED MEDCORP RELATION COUNTS

Structure	Rows
Legacy records	100,000
Patients	19,893
Doctors	50
Wards	10
Treatments	50
Appointments	99,999
Appointment-treatment links	100,000

III. METHODOLOGY

A. Dataset Profiling

Python profiling identified 19,893 patients, 50 doctors, 10 wards, and 50 treatment codes. The source contained no duplicate complete rows and no single-column candidate key. The main dependencies were:

B. Candidate Keys and Reconstruction

No single attribute uniquely identified a legacy row. The combination of Patient_ID, Appointment_Date, Doctor_ID, and Ward_ID produced 99,999 appointment groupings. Adding Treatment_Code produced 100,000 unique legacy records. The original treatment-occurrence key was:

$$K = (\text{Patient_ID}, \text{Appointment_Date}, \text{Doctor_ID}, \text{Ward_ID}, \text{Treatment_Code}) \quad (1)$$

The original file did not contain an appointment identifier. The reconstruction script preserved source order, grouped rows by the four appointment attributes, and generated surrogate identifiers such as A000001. This separated appointment identity from treatment occurrence without discarding any of the 100,000 source rows.

C. Normalization Proof

The CSV is treated as 1NF because each field is atomic and each row represents one appointment-treatment occurrence. The relation violates 2NF because patient, doctor, ward, and treatment attributes depend on proper subsets of the composite key K . Decomposition places each determinant and its attributes in a separate relation: Patient, Doctor, Ward, and Treatment. Appointment stores the patient, doctor, ward, date, and generated appointment key; AppointmentTreatment stores the composite association between appointments and treatments.

TABLE III
FLAT VERSUS NORMALIZED BENCHMARK RESULTS

Metric	Flat	Normalized	Unit
Memory footprint	113.056	93.088	MB
Doctor mutation median	3.807	0.0001	ms
1,000-patient read median	4.024	2.264	ms

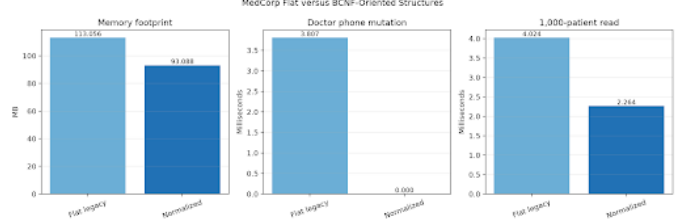


Fig. 2. Comparison of flat legacy and BCNF-oriented Python structures.

The 3NF condition is satisfied because non-key attributes do not depend transitively on other non-key attributes. For BCNF, every determinant is a superkey in its relation: Patient_ID, Doctor_ID, Ward_ID, Treatment_Code, and Appointment_ID are relation keys. AppointmentTreatment contains only its composite key and foreign keys, so it has no non-key dependency.

D. Benchmark Design

The flat representation was a list of 100,000 dictionaries. The normalized representation used dictionaries for patients, doctors, wards, treatments, and appointments, plus indexes for appointment-treatment lookups. Each measurement was repeated 30 times, and median values were reported. The benchmark measured approximate memory footprint, doctor-phone mutation time, and reconstruction latency for records associated with 1,000 patients. It measures Python data structures rather than a production SQL or distributed database.

IV. RESULTS

The normalized structure reduced the approximate memory footprint from 113.056 MB to 93.088 MB, a reduction of approximately 17.7%. The reduction resulted from storing descriptive patient, doctor, ward, and treatment attributes once rather than repeating them across the legacy records.

Updating doctor D004 required scanning all 100,000 flat records and changing every matching row. The normalized structure changed one doctor dictionary entry, producing a near-zero measured median. The normalized read path also performed better in this implementation: 2.264 ms compared with 4.024 ms for the flat scan.

V. DISCUSSION

The results demonstrate the integrity benefit of normalization. A doctor phone-number update is performed once in the normalized Doctor structure rather than across potentially thousands of redundant legacy rows. This directly addresses

the update anomaly and reduces the probability of inconsistent values.

The read result requires careful interpretation. The normalized representation did not produce higher latency because it used indexed Python dictionaries, while the flat representation performed a linear scan through 100,000 rows. Thus, the cost of programmatic joins was offset by targeted lookups. This does not prove that normalized designs always produce faster reads; distributed joins across machines or partitions may be substantially more expensive than in-memory dictionary lookups.

Architectural compromise. A practical compromise for read-heavy systems is controlled denormalization through materialized views or read-optimized projections. The BCNF tables remain the authoritative write model, while a derived read model combines commonly accessed patient, appointment, doctor, ward, and treatment fields. This preserves normalized write behavior while reducing repeated joins for frequent reads. Recent schema-transformation research supports treating denormalization as a workload-dependent design choice [2], [3].

Limitations. The benchmark measures Python in-memory structures rather than a production database. Memory values are approximate `sys.getsizeof()` estimates, and the normalized read test uses pre-built indexes while the flat test performs a linear scan. The experiment uses one dataset, one doctor mutation, a 1,000-patient read workload, and 30 repetitions. It does not measure disk I/O, network latency, transaction contention, concurrent users, or cross-node joins. The results should therefore be interpreted as workload-specific evidence rather than universal database performance claims.

VI. CONCLUSION

The MedCorp legacy dump contained substantial redundancy because patient, doctor, ward, and treatment attributes were repeated across appointment-treatment records. Functional-dependency analysis justified decomposing the flat relation into Patient, Doctor, Ward, Appointment, Treatment, and AppointmentTreatment relations. The generated `Appointment_ID` provided a practical surrogate key for appointment-level records that were not explicitly identified in the source data.

The BCNF-oriented design reduced the measured Python memory footprint by approximately 17.7% and made doctor-phone mutation substantially faster by replacing a full-table update with a direct entity lookup. The normalized read path was also faster for the tested 1,000-patient workload because dictionary indexes offset the cost of reconstructing joined records. These results demonstrate that normalization improves integrity and update behavior, while read performance depends on physical implementation and workload. For read-heavy systems, an authoritative normalized write model combined with controlled denormalized projections provides a practical compromise between consistency and access speed.

CODE AVAILABILITY

The reconstruction, benchmarking, and visualization scripts are available in the project repository:

[https:](https://github.com/zacfrappier/medcorp-normalization-benchmark)

[//github.com/zacfrappier/medcorp-normalization-benchmark](https://github.com/zacfrappier/medcorp-normalization-benchmark)

REFERENCES

- [1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970, doi: 10.1145/362384.362685.
- [2] J. Mali, S. Ahvar, F. Atigui, A. Azough, and N. Travers, "A global model-driven denormalization approach for schema migration," in *Research Challenges in Information Science*, Lecture Notes in Business Information Processing, vol. 446, Springer, 2022, pp. 529–545, doi: 10.1007/978-3-031-05760-1_31.
- [3] N. Bansal, S. Sachdeva, and L. K. Awasthi, "Query-based denormalization using hypergraph (QBDNH): A schema transformation model for migrating relational to NoSQL databases," *Knowledge and Information Systems*, vol. 66, pp. 681–722, 2024, doi: 10.1007/s10115-023-02017-y.